COMPUTER SCIENCE + COMPUTER ENGINEERING FOR GAMES

PACKAGE 07 / VOLUME 06

Module 29: Multiplayer simulation and online services

Four expanded lessons • Project and review

UPPER UNDERGRADUATE
Modules 26-30 · Lessons 26.1-30.4 · Gate F

Edition: September 28, 2026

Original expanded instruction, worked examples, quizzes, implementation contracts and evidence-based review. The learner performs the projects and platform/human validation. This self-study curriculum is not an accredited degree.

Contents

Module 29: Multiplayer simulation and online services 4

Lesson 29.1: Authority, prediction and reconciliation . . . 5

Lesson 29.2: Snapshots, interpolation, relevance and lag compensation . . . 8

Lesson 29.3: Lockstep, rollback and deterministic replay . . . 11

Lesson 29.4: Match services, identity, persistence and failure recovery . . . 14

Module 29 project . . . 16

Module 29 review . . . 17

Reference Directory 18

S01: Collision documentation . . . 18

S02: Simulation documentation . . . 18

S03: glTF tutorial: Skins . . . 18

S04: glTF 2.0 specification . . . 18

S05: Writing a callback function . . . 18

S06: SDL_AudioStreamCallback . . . 18

S07: Xbox Accessibility Guidelines . . . 18

S08: CS188: Informed search . . . 19

S09: Introduction . . . 19

S10: CS188: Minimax . . . 19

S11: Automatic differentiation with torch.autograd . . . 19

S12: Networked movement in the Character Movement Component . . . 19

S13: Snapshot Interpolation . . . 19

S14: GGPO SDK repository 19

S15: Transaction documentation 20

S16: Atomic Commit in SQLite 20

S17: os.replace documentation 20

S18: zipfile documentation 20

S19: Fix Your Timestep! 20

S23: XAG 103: Additional channels for audio and visual cues 20

S24: Deterministic Lockstep 20

S25: XAG 107: Input 20

S26: XAG 101: Text display 21

Original program reading IDs 21

Module 29: Multiplayer simulation and online services

Prerequisites: Modules 18-20 and 24; Module 26 for physical interaction.

Original reading IDs: R17, R20, R26. See the reference directory.

Module goal: Create a local two-client multiplayer demonstration with a network simulator for delay, jitter, loss, duplication and reordering. No public server or paid cloud account is required. Document authority and recovery rules.

Lesson 29.1: Authority, prediction and reconciliation

Learning goal: Provide responsive local control without trusting arbitrary client outcomes.

Decide who owns truth

An authoritative server accepts player intentions and computes permitted state transitions. A client can request movement; it cannot simply declare a new position, reward total or health value and expect trust. Authority is a game-design and security boundary. The local lab uses synthetic clients and no public service, but it must still validate inputs and ownership.

Define a message schema with protocol version, session/epoch, player identity, input sequence and bounded control values. The server chooses the simulation timestep. A client-supplied dt can otherwise become a speed multiplier. Authentication establishes a principal; authorization decides which actor that principal may control. This lesson's local harness models those roles without claiming to implement production identity or encryption.

Prediction hides response delay

A client immediately simulates its own legal input using the same movement rule as the authority. It retains unacknowledged inputs. The server later reports an authoritative state and the highest contiguous input sequence included in that state. The client restores that state, removes acknowledged inputs and replays the remaining ones in sequence. This is reconciliation. Epic's networked-movement documentation is a concrete engine comparison [S12].

The acknowledgment must identify exactly which input effects the state contains. The largest sequence number merely received is not enough when earlier messages are missing. Duplicate inputs must not be applied twice. A server can buffer out-of-order commands within a bound or use an explicit late-input policy. It must not leave an unbounded queue waiting forever for a missing packet.

Worked acknowledgment trace

The server sends x=10 after processing input 40. The client has predicted inputs 41,42,43,44, each moving +1 under this teaching rule. Reconciliation sets predicted x to 10 and replays those four inputs, yielding 14. Replaying input 40 again would move to 15 and double-apply acknowledged work. Snapping to 10 without replay would discard valid pending input and feel unresponsive.

Now suppose input 43 attempts an illegal +50 move while legal inputs are -1,0,+1. The server rejects or substitutes a neutral input according to its contract. The client may have predicted incorrectly, so a correction is expected. Validation must be performed before the authoritative update; merely clamping the final rendered position does not repair the server's trusted state.

Logical correction versus visual smoothing

The authoritative correction must affect future simulation immediately at a known tick. Visual smoothing can gradually reduce a render-only offset so the player does not see a harsh jump. Do not smooth the logical collision state toward authority while continuing to predict from the wrong state; that can create persistent divergence. Keep the simulation position and presentation offset separate.

Prediction becomes harder with interacting physics, nondeterministic code and shared objects. A simple movement example does not establish full networked rigid-body correctness. Start with one-dimensional or grid movement, then state which collisions and interactions are predicted and which wait for authority. A mismatch between client and server movement code is a correctness bug, not ordinary network jitter.

Transport faults and recovery

Build a seeded delivery queue that can delay, drop, duplicate and reorder messages. Keep a bounded command history on the client. If an acknowledgment is older than the last applied authoritative snapshot, ignore it under the protocol's sequence comparison rule. The teaching code uses unbounded Python integers for sequence numbers; a production fixed-width protocol requires an explicit wraparound comparison window.

Persistent packet loss cannot be defeated by finite retries with a guaranteed delivery claim. Define a retry budget and a resynchronization or disconnect state. To test convergence, stop generating new inputs and enter a declared reliable drain phase. Then both sides should agree on all accepted input effects. Label that phase rather than implying arbitrary permanent loss always converges.

Deliberate practice

The kernel reconciles an acknowledged state with a sorted, validated pending input list. The full lab runs two distinct clients and an authority through fault injection, records corrections and validates ownership. Include duplicates, a missing middle sequence, stale snapshots, invalid movement and reconnect with a new epoch. Report the final accepted input set and state, not only an animation of two dots.

In the engine, input capture produces commands; simulation consumes commands; networking transports them; presentation displays predicted or interpolated snapshots. Keep those layers independent so a local replay can reproduce a network bug without a live server. A paid cloud account is unnecessary for this learning objective.

Original worked checkpoint

The server acknowledges input 40. The client corrects to the returned state for that point and replays locally stored inputs 41 through 44, rather than applying those inputs twice to the wrong starting state.

Full practical assignment

Implement a one-dimensional or grid movement server with client prediction. Pass: invalid movement is rejected and induced delay produces documented corrections without permanently diverging state.

The complete procedure, failure matrix and acceptance evidence are in the corresponding LAB_WORKBOOK card. The small kernel is **Input acknowledgment and reconciliation**. Run `starter/python/demos/lesson_29_1.py`; implement `starter/python/exercises/lesson_29_1.py`.

Assigned study and native transfer

R17 : transport semantics; R26 : networking and multiplayer concepts; R20 : networking in games.

Read S12 on client prediction, saved moves and correction. Compare the concepts rather than assuming the teaching protocol reproduces Unreal internals. Revisit Modules 18-20 for concurrency and trust boundaries.

Engine transfer: Create an engine command schema and separate authoritative, predicted and render-only state. Record one correction with the exact acknowledged and replayed input IDs.

Check your understanding

1. What is authoritative state?
2. Which inputs are replayed after acknowledgment of input 40 in the example?
3. Why send intentions rather than unrestricted reward totals?
4. Authority acknowledges input 40 at x=10. Pending inputs 41 through 44 each move +1. What is the reconciled prediction?
5. Why is acknowledging the highest received sequence unsafe when an earlier input is missing?

Attempt all five before opening the answer key. A passing quiz alone does not complete the lesson.

Lesson 29.2: Snapshots, interpolation, relevance and lag compensation

Learning goal: Manage bandwidth and latency while making fairness assumptions explicit.

Remote actors are usually presented in the past

A local actor can be predicted from its inputs. A remote actor's future intentions are unknown. Snapshot interpolation buffers recent authoritative states and renders a time slightly behind the newest known server time. This trades extra presentation delay for smoother behavior under jitter. The original networking article by Glenn Fiedler is the assigned conceptual comparison [S13].

Use server ticks or a mapped server-time estimate, not packet arrival time, as the interpolation axis. Arrival time includes variable transport delay. Let snapshots bracket render time t_r at t0 and t1. With t0<t1, alpha=(t_r-t0)/(t1-t0), and a linearly interpolated position is (1-alpha)*x0*+*alpha*x1. Orientation needs the rotation interpolation rules from animation.

Worked timeline

Snapshots at server times 1.00 s and 1.10 s contain x=2 m and x=4 m. Rendering at 1.04 s gives alpha=0.4 and x=2.8 m. If the newest estimated server time is 1.14 s, a 0.10 s interpolation delay leads to that 1.04 s render time. The transport may have delivered both snapshots almost simultaneously; their arrival spacing must not replace their simulation spacing.

When no later snapshot exists, choose a bounded extrapolation or hold policy. Extrapolation can improve brief gaps but may move an actor through a wall or around a turn it never took. A teleport or respawn is a discontinuity and should reset or segment the interpolation history. Never interpolate across different entity generations or match epochs.

Ordering and bounded history

Receive snapshots into a timestamped, deduplicated structure. A late packet may still be useful if it fills the current interpolation bracket; a packet older than the retained window can be discarded. Decide whether your simpler implementation retains out-of-order samples or accepts only increasing snapshots. Both require explicit consequences and tests. Bound memory, timestamps and entity counts before accepting untrusted data.

The kernel sorts unique timestamp/position samples, rejects conflicting duplicates and holds endpoints outside the available interval. The full lab must add a missing-snapshot policy, an interpolation-delay setting and a graph or table of perceived delay versus correction/jitter behavior. A smooth result can still have unacceptable latency, so measure both.

Delta compression requires a known baseline

A delta snapshot describes changes relative to a particular baseline ID. The receiver must possess that exact baseline. Guessing the most recent available snapshot can reconstruct plausible but wrong state. If the baseline is missing, request or await a full state according to the recovery policy. Retain baselines only within a bounded acknowledgment window. Compression is an optimization layered on a correct state protocol.

Calculate bandwidth from actual serialized bytes, snapshot rate and recipients. For example, 40 bytes per entity, 100 relevant entities and 20 snapshots/s produce 80,000 bytes/s of payload per client before headers and retransmission. Do not describe that as the total server bandwidth without multiplying by clients and including protocol overhead.

Relevance is also an information boundary

Do not send hidden enemy state merely because the normal renderer chooses not to display it. A modified client can inspect received data. Determine relevance on the authority using a visibility or gameplay policy, then serialize only allowed fields. Relevance and compression solve different problems: one selects what may be known; the other encodes it efficiently. A spectator may have different permissions from a player.

Lag compensation re-evaluates an action against a bounded historical world to account for delay under a game-specific fairness policy. A client's claimed timestamp is not unlimited permission to rewind. Bound the rewind horizon, validate timing plausibility and document whose experience is favored. The core lab need not implement a full shooter hit-registration system, but it must explain this tradeoff accurately.

Deliberate practice

Add a second remote actor to the network harness, interpolate its snapshots and filter a hidden entity from one client. Capture serialized output to prove exclusion before rendering. Inject out-of-order snapshots, a missing baseline and a teleport. Report buffer delay, held/extrapolated frames, dropped history and full-state recovery. In your engine, keep remote presentation state separate from the collision authority and from local prediction.

Original worked checkpoint

Buffering two snapshots 50 ms apart can interpolate motion during a delay variation, but it intentionally displays a slightly older world. This is a latency-quality tradeoff, not free smoothing.

Full practical assignment

Add remote interpolation and relevance filtering. Pass: out-of-order snapshots are handled, missing baselines trigger recovery, and the report explains both visual delay and hidden-state exposure policy.

The complete procedure, failure matrix and acceptance evidence are in the corresponding LAB_WORKBOOK card. The small kernel is **Timestamped snapshot interpolation**. Run `starter/python/demos/lesson_29_2.py`; implement `starter/python/exercises/lesson_29_2.py`.

Assigned study and native transfer

R20 : networked game state; R26 : replication and network smoothing; revisit Module 19 message validation.

Read S13 on interpolation buffers and linear interpolation. Revisit S12 for an engine comparison. Derive the payload bandwidth of your actual schema.

Engine transfer: Implement authority-side relevance filtering and capture the outgoing payload. Demonstrate that a hidden entity is absent from the unauthorized client, not merely invisible in the renderer.

Check your understanding

1. What cost does snapshot interpolation commonly add?
2. Why must delta-compression baselines be identified?
3. Why avoid transmitting irrelevant hidden enemy state?
4. Positions are 2 m at 1.00 s and 4 m at 1.10 s. What is the interpolated position at 1.04 s?
5. A delta names baseline 120, but the receiver only retains baseline 121. Should it apply the delta to 121?

Attempt all five before opening the answer key. A passing quiz alone does not complete the lesson.

Lesson 29.3: Lockstep, rollback and deterministic replay

Learning goal: Choose synchronization around the game model rather than genre slogans.

Input lockstep and rollback make different latency choices

Lockstep advances a tick only after the required input bundle is known. When all participants run the same deterministic transition, exchanging inputs can be cheaper than exchanging full state. Waiting for delayed input can stall play. Rollback predicts missing input, saves earlier states, and re-simulates when the actual input arrives. GGPO is an original rollback SDK reference, not a claim that this package integrates it [S14].

Define the state boundary before implementing rollback. Let S_t be the state before tick t and I_t the input for that tick. Save S_t, predict or receive I_t, then compute S_(t+1). If input for tick k changes, restore S_k and re-run ticks k through the latest completed tick. Restoring S_(k+1) instead is an off-by-one error that preserves the effect of the wrong input.

What must be saved

A snapshot needs every future-affecting deterministic field: positions, velocities, gameplay timers, entity generations, random state, spawn counters, AI state and necessary solver caches. External pointers are not portable state. A copied array of visible transforms is usually insufficient. Version the schema and test the next tick after restore, not just the restored values.

At 60 Hz, 120 saved ticks cover two seconds. With 2 MB per state, raw snapshots alone consume 240 MB under decimal MB units. Metadata, input history and event records add more. Compression or incremental state can reduce storage, but also adds restore cost and failure modes. Measure the complete history budget rather than quoting the smallest structure in isolation.

Worked replay correction

A scalar simulation starts S_0=0 and consumes inputs [1,1,0,1]. Predicted post-step states are 1,2,2,3. Later, the actual input at tick 2 is -1 rather than 0. Restore S_2=2, apply -1 to obtain S_3=1, then apply tick 3's +1 to obtain S_4=2. The corrected final state is 2. Replaying all inputs from a known initial state is a useful independent oracle for the small test.

Now add an event when the state first reaches a target. Re-simulation may regenerate the same event. Presentation deduplicates by stable identity; durable rewards wait for an appropriate confirmation boundary. Sending a payment, file write or external message directly from a replayed tick creates repeated irreversible side effects. Separate deterministic event generation from external commitment.

Determinism is still required within the chosen scope

Fixed dt alone does not guarantee deterministic replay. Unstable pair ordering, unseeded randomness and excluded caches can diverge. Use the first-divergence harness from Lesson 26.4. Deliberately change one operation or input and verify that the report identifies the earliest differing state. Glenn Fiedler's lockstep article is a useful description of the assumptions and buffering tradeoffs [S24].

A late input older than retained history cannot be repaired by restoring a state that no longer exists. Define a recovery policy: request full authoritative state, pause for resynchronization, or disconnect under a clear error. Never silently apply the old input to the current tick. Bound the maximum rollback depth and re-simulation work per frame. A large correction can exceed the frame budget even when each ordinary tick is fast.

Test side effects, not just positions

Run a delay scenario that forces repeated replay across the same event tick. Count deterministic event generations, unique presentation events and committed durable effects separately. It is normal for deterministic generation to occur several times during replay; it is not normal for a reward to be committed several times. Keep the deduplication cache outside the restored simulation snapshot.

Use a stable match epoch so a new match can legitimately generate an event with the same tick and entity number. Retain only the event history needed for the rollback window, plus any durable idempotency records whose lifetime belongs to a different system.

Deliberate practice

The kernel replays an integer input tape after a correction and returns the corrected state sequence. The full lab adds a bounded snapshot ring, delayed input, first-divergence hashes and side-effect suppression. Compare a corrected run to a run that knew all inputs from the start. Include the oldest retained tick, an input one tick too old, duplicate late input and a changed random seed. Submit a timing and memory report for worst observed re-simulation depth on the declared environment.

Original worked checkpoint

At 60 simulation ticks per second, storing 120 snapshots covers two seconds of history. If each snapshot is 2 MB, raw snapshot storage alone reaches 240 MB before metadata or compression.

Full practical assignment

Implement rollback or input lockstep for the tiny deterministic game. Pass: a delayed-input test converges, state hashes detect a deliberately introduced divergence, and audio or reward side effects are not duplicated.

The complete procedure, failure matrix and acceptance evidence are in the corresponding LAB_WORKBOOK card. The small kernel is **Rollback to the pre-tick state**. Run `starter/python/demos/lesson_29_3.py`; implement `starter/python/exercises/lesson_29_3.py`.

Assigned study and native transfer

R20 and R26 : multiplayer concepts; derive the implementation using Modules 18, 24 and 26 rather than assuming engine-specific guarantees.

Read the S14 SDK overview and S24 on deterministic lockstep. Reuse the serialization and first-divergence evidence from Lesson 26.4; do not begin from a transform-only snapshot.

Engine transfer: Add a bounded rollback ring to your engine or a separate deterministic two-client demonstration. Keep irreversible side effects behind an explicit confirmation/idempotency boundary.

Check your understanding

1. How much time do 120 ticks cover at 60 Hz?
2. How much raw storage do 120 snapshots of 2 MB require?
3. Why are external side effects difficult during rollback?
4. With S_0=0 and predicted inputs [1,1,0,1], tick 2 is corrected to -1. Which state must be restored and what is the corrected S_4?
5. A late input refers to a tick older than the snapshot ring. What is a valid response, and what is an invalid shortcut?

Attempt all five before opening the answer key. A passing quiz alone does not complete the lesson.

Lesson 29.4: Match services, identity, persistence and failure recovery

Learning goal: Design a small online service without treating it as part of the render loop.

Online services are state machines with failures

A match registry might move through created, joining, active, completing and closed states. Write allowed transitions and which principal may trigger each. Authentication says who is making a request; authorization says whether that principal may act on this match. A client-supplied player name is not authentication. The local lab uses synthetic principals provided by a trusted test harness and is not a production identity service.

A service request can fail before processing, after committing but before the reply, or while the caller times out without knowing the outcome. Retrying is necessary, but retrying a reward operation naively can duplicate its effect. The important question is not whether the network delivered a message exactly once; it is whether the durable effect occurs once under the defined retry contract.

Idempotency needs more than a set in memory

Assign a stable operation ID, such as a match-result ID, and bind it to the canonical payload. On first valid submission, record the operation and apply its effect in one database transaction. On an exact retry, return the prior outcome without applying the effect again. If the same ID arrives with a different payload, reject the conflict rather than silently accepting a different result.

The operation record and the reward update must commit together. Recording the ID first and then crashing before the reward can permanently lose the effect. Applying the reward first and crashing before recording the ID can duplicate it on retry. SQLite's transaction and atomic-commit documentation explains the durable transaction model used for the local comparison [S15, S16]. Actual durability also depends on configuration, storage and failure assumptions.

Worked timeout after commit

The authority submits result R17 awarding 50 points to player P. The store starts a transaction, inserts R17 with its payload, increments P's points and commits. The response is lost. The authority retries R17. The store finds the same payload and returns 'already applied' with no additional points. P has 50, not 100. A retry of R17 awarding 500 points is a conflict and must fail.

Now restart the store process between submissions. An in-memory set forgets R17; a durable operation table does not. The lesson therefore requires a file-backed local database test, not only a dictionary. Parameterize SQL inputs and validate numeric ranges. Do not construct a query by concatenating an untrusted name or operation ID.

Authorization before mutation

Only a trusted match authority should commit its result. A player may read permitted match information but must not edit another player's reward. Check that the submitting principal owns the active authority lease or meets the lab's simpler explicit role rule. Re-check necessary state within the transaction to avoid a stale authorization decision racing with a state change.

For a production system, tokens, key rotation, transport security, abuse controls and account recovery would be additional work. They are intentionally outside this bounded local lab. Do not expose its synthetic identity switch on a public network or describe it as secure authentication.

Bounded retry and recovery

Use a finite retry budget with backoff and a user-visible pending or unavailable state. Retry only operations that are safe under the protocol. A permanent validation error is not repaired by retrying forever. Distinguish timeout, authorization failure, conflict and unavailable storage in both code and user feedback.

A lease or heartbeat can help detect an abandoned match, but absence of a heartbeat is uncertainty rather than proof that the other process has stopped. Avoid two simultaneous authorities by defining lease ownership and version checks. The core assignment can use a single local authority and still document what would need to change for multiple processes.

Deliberate practice

The kernel validates principal, payload and idempotency against an in-memory model. The integration companion uses SQLite to test commit, exact retry, conflict, unauthorized access and restart persistence. The full lab adds a match-registry state machine and service-loss behavior. Inject a failure immediately before commit and immediately after commit, then retry with the same operation ID.

Submit a state-transition table, operation schema, test logs and a client outcome for each failure class. Keep saves and local player progress separate from temporary match state. No paid cloud, real accounts, payment processing or public deployment is required. The engineering objective is a coherent failure-aware contract, not a large online platform.

Original worked checkpoint

A match-result submission is retried after a timeout. A stable operation identifier lets the progression service recognize the duplicate and avoid granting the same reward twice.

Full practical assignment

Build a local match registry and idempotent result store. Pass: duplicate submissions do not duplicate rewards, unauthorized edits fail, and simulated service loss produces a documented client outcome.

The complete procedure, failure matrix and acceptance evidence are in the corresponding LAB_WORKBOOK card. The small kernel is **Idempotent result-store model**. Run `starter/python/demos/lesson_29_4.py`; implement `starter/python/exercises/lesson_29_4.py`.

Assigned study and native transfer

R18 from the directory: transactions; R17 : transport; revisit Modules 19-20 for distributed failure and threat modeling.

Read S15 on transactions and S16 on atomic commit. Review Module 20 trust boundaries. Treat the supplied synthetic-principal code as a local test model, not production authentication.

Engine transfer: Implement a file-backed local result store and prove exact retry behavior across a process restart. Add a match-state transition table and an unavailable-service client state.

Check your understanding

1. Is authentication the same as authorization?
2. Why identify a match-result operation uniquely?
3. Must this lab use a paid public cloud?
4. A result commits, but its response is lost. The client retries with the same operation ID and payload. What must happen to the reward total?
5. Why must a reused operation ID with a different payload be rejected instead of treated as a normal retry?

Attempt all five before opening the answer key. A passing quiz alone does not complete the lesson.

Module 29 project

Build a Local Two-Client Simulation. Use one authoritative process and two local clients, or an equivalent process/socket layout with explicit identities and controlled inputs. The package's in-process simulator is a reference for message semantics, not a substitute for the student's transport adapter. No public endpoint or paid service is required.

Exercise delay, jitter, loss, duplication and reordering with seeded schedules. Include local prediction/reconciliation, a remote interpolation buffer and one missing-baseline recovery. Show which state is omitted by relevance filtering. Use rollback or input lockstep for a tiny deterministic mode, with a documented history/barrier policy and side-effect handling.

Add a local registry and transactional result store. Inject failure before and after commit and restart the service. Keep synthetic test identities clearly distinct from production authentication. Demonstrate an explicit client state while the service is unavailable rather than silently losing results.

Deliver protocol schemas, process topology, fault schedules, correction/replay traces, hidden-state payload audit, transaction/restart logs and limitations. Review asks what is authoritative, what is predicted, what can be replayed and what must be idempotent.

Module 29 review

Answer these after the project work. They are additional to the 20 lesson questions in this module.

1. The server receives inputs 10, 12 and 13 but not 11. What may it acknowledge under the contiguous-ACK protocol?
2. Is hiding an entity in the renderer sufficient to prevent clients from learning its position?
3. Explain the off-by-one error in restoring S_(t+1) when replacing input I_t.
4. A reward committed but the response was lost. How does the client retry without duplicating the reward?
5. What does an in-process faulty-network test prove, and what remains to test?

Reference Directory

S-prefixed entries are the focused primary-source companions consulted for this expansion, September 28, 2026. Numbers are stable identifiers; gaps are intentional. R-prefixed entries preserve the original program reading directory. External books, papers, documentation and library source are not bundled; the explanations, exercises and demonstration code in this package are original teaching material. Pin installed versions in your own evidence rather than treating a mutable documentation page as an installed build.

S01: Collision documentation

Box2D project. Shape queries, broad phase, dynamic tree and time-of-impact limitations.

Open primary source

S02: Simulation documentation

Box2D project. Stepping, contacts, solver settings and the documented determinism contract.

Open primary source

S03: glTF tutorial: Skins

Khronos Group. Joint transforms, inverse bind matrices and mesh-node correction.

Open primary source

S04: glTF 2.0 specification

Khronos Group. Normative asset, animation and skinning data contracts.

Open primary source

S05: Writing a callback function

PortAudio project. Callback constraints and real-time-safe execution discipline.

Open primary source

S06: SDL_AudioStreamCallback

SDL project. SDL3 audio stream callback contract; consult the API for the chosen integration.

Open primary source

S07: Xbox Accessibility Guidelines

Microsoft. Game accessibility design/testing guidance, not a legal-compliance certification.

Open primary source

S08: CS188: Informed search

University of California, Berkeley. A*, heuristics and search guarantees under stated assumptions.

Open primary source

S09: Introduction

BehaviorTree.CPP project. Behavior-tree concepts, statuses and implementation context.

Open primary source

S10: CS188: Minimax

University of California, Berkeley. Adversarial search and alpha-beta reasoning.

Open primary source

S11: Automatic differentiation with torch.autograd

PyTorch project. Gradient/parameter concepts; framework use is optional for the core synthetic exercise.

Open primary source

S12: Networked movement in the Character Movement Component

Epic Games. A maintained engine comparison for authority, prediction and correction.

Open primary source

S13: Snapshot Interpolation

Glenn Fiedler. Author’s technical explanation of interpolation, jitter and visual delay.

Open primary source

S14: GGPO SDK repository

GGPO authors. Primary implementation/documentation for rollback networking concepts.

Open primary source

S15: Transaction documentation

SQLite project. Transaction boundaries and behavior.

Open primary source

S16: Atomic Commit in SQLite

SQLite project. Atomicity assumptions and storage failure considerations.

Open primary source

S17: os.replace documentation

Python Software Foundation. Replacement semantics and platform/filesystem conditions.

Open primary source

S18: zipfile documentation

Python Software Foundation. Archive processing and extraction boundaries.

Open primary source

S19: Fix Your Timestep!

Glenn Fiedler. Simulation/render time separation and work-budget hazards.

Open primary source

S23: XAG 103: Additional channels for audio and visual cues

Microsoft. Critical information through more than one channel.

Open primary source

S24: Deterministic Lockstep

Glenn Fiedler. Input synchronization and deterministic simulation requirements.

Open primary source

S25: XAG 107: Input

Microsoft. Input/remapping accessibility considerations.

Open primary source

S26: XAG 101: Text display

Microsoft. Readable interface text and testing considerations.

Open primary source

Original program reading IDs

These inherited assignments remain part of the full study route. S entries above focus the expanded lesson tasks; they do not replace all external study.

R17: RFC 9293: Transmission Control Protocol — IETF / RFC Editor. Transport semantics; reference reading after networking foundations. Assigned source

R19: 15-462/662 Computer Graphics — Carnegie Mellon University. Rendering, geometry and animation course structure and exercises. Assigned source

R20: 15-466/666 Computer Game Programming — Carnegie Mellon University. Runtime systems and game-production programming. Assigned source

R22: SDL3 documentation — SDL project. Windowing, input, audio and platform services; do not mix SDL2 and SDL3 examples. Assigned source

R23: Physically Based Rendering, fourth edition — Matt Pharr, Wenzel Jakob, Greg Humphreys. Rendering mathematics and implementation; use chapter titles in lesson assignments. Assigned source

R26: Programming with C++ in Unreal Engine — Epic Games. Engine comparison after C++ foundations, not a beginner substitute. Assigned source

R28: Box2D documentation — Box2D project. Collision, constraints and simulation reference. Assigned source

R29: Xbox Accessibility Guidelines — Microsoft. Accessibility design and testing; not a legal-compliance certificate. Assigned source

R45: AddressSanitizer documentation — LLVM Project. Memory-error instrumentation, limitations and supported toolchains. Assigned source

R46: CS188 Introduction to Artificial Intelligence textbook — University of California, Berkeley. Search, games, decision processes, learning and probabilistic reasoning. Assigned source

R47: Dive into Deep Learning — Aston Zhang, Zachary Lipton, Mu Li and Alexander Smola. Learning fundamentals, gradients, generalization and reproducible exercises; pin compatible framework versions. Assigned source